

2026학년도 전기 졸업과제 중간보고서 (분과 A 소프트웨어/인공지능)

몰입형 XR을 위한 상황 인식 기반 Adaptive Passthrough Framework

Context-aware Adaptive Passthrough Framework for Immersive XR



팀명: Team VR

202355532 따다소 thadarsoe730@pusan.ac.kr

202355667 최아영 ayoun0421@pusan.ac.kr

202155583 이승주 maya0108@pusan.ac.kr

지도교수: 이명호 교수님

제출일: 2026년 07월 12일

부산대학교

목차

1	서론	1
1.1	프로젝트 소개	1
1.2	프로젝트 배경 및 필요성	1
1.3	프로젝트 목표	2
2	전체 시스템 개요	3
2.1	시스템 처리 흐름 및 위험도	3
2.2	시스템 레이어 정의	4
3	요구조건 및 제약 사항 분석에 대한 수정사항	5
3.1	정적 경계 및 사용자 상태 설계에 대한 수정사항	5
3.1.1	정적 경계 데이터	5
3.1.2	위험도 명칭 변경	5
3.2	동적 객체 탐지 및 위험도 설계에 대한 수정사항	6
3.2.1	구현 범위와 출력의 재정의	6
3.2.2	데이터 정의와 측정 제약	6
3.2.3	Feature 선정 근거	7
3.3	로그 기반 개인화 시스템 설계 수정사항	7
3.3.1	저장 세션 로그 항목 수정 및 Feature 재정의	7
4	설계 상세화 및 변경 내역	9
4.1	정적 경계 및 사용자 상태 설계 상세화	9
4.1.1	정적 경계 위험도 설계	9
4.1.2	사용자 상태 위험도 설계	10
4.2	동적 객체 탐지 및 위험도 설계 상세화	10
4.2.1	모듈 구조와 처리 흐름	10
4.2.2	규칙 기반 위험도 설계	11

4.3	로그 기반 개인화 시스템 설계 상세화	12
4.3.1	로그 수집 및 버퍼 적재 및 Feature 변환	12
4.3.2	실버 라벨링 및 온디바이스 모델 학습	12
4.3.3	연속적 파라미터 매핑 및 실시간 피드백	13
5	프로젝트 진척도 및 수행 내용	14
5.1	구성원별 진척도	14
5.1.1	최아영	14
5.1.2	이승주	15
5.1.3	따다소	15
5.2	보고 시점까지의 과제 수행 내용 및 중간 결과	16
6	갱신된 과제 추진 계획	18
6.1	시스템 통합 및 성능 개선	19
6.2	비교 평가 계획	19

제1장 서론

1.1 프로젝트 소개

본 프로젝트는 Meta Quest 기반의 몰입형 Extended Reality(XR) 환경에서 사용자의 안전과 몰입감을 동시에 확보하기 위한 **상황 인식 기반 Adaptive Passthrough Framework** 프로토타입 개발을 목표로 한다. Head Mounted Display(HMD)의 카메라와 센서로 사용자 및 주변 환경 정보를 수집하고, 정적 경계와 동적 객체의 위험을 실시간으로 분석하여 상황에 적절한 수준의 Passthrough 시각화를 제공한다.

기존 경계 시스템처럼 현실 화면을 단순히 켜고 끄는 방식에서 벗어나 위험 방향과 위험 수준에 따라 현실 정보의 범위와 표현 방식을 조절하는 것이 핵심이다. 또한 주변 사람, 반려동물 및 이동 물체와 같이 시간에 따라 위치가 변하는 동적 객체를 탐지하고, 사용자와 객체 사이의 상대적 움직임을 분석하여 잠재적인 충돌 위험을 사전에 전달하고자 한다. 장기적으로는 Passthrough 작동 이력과 사용자 반응 로그를 활용하여 사용자마다 다른 공간 환경과 행동 패턴에 적응하는 개인화 제어 구조로 확장한다.

1.2 프로젝트 배경 및 필요성

몰입형 HMD는 가상 환경에 대한 높은 몰입감을 제공하지만 사용자가 실제 주변 공간과 사람을 직접 확인하기 어렵게 만든다. 상용 HMD의 기존 Guardian 또는 Boundary 시스템은 사용자가 사전에 설정한 활동 경계에 접근하면 경고나 Passthrough를 제공한다. 그러나 이러한 방식은 고정된 경계를 중심으로 동작하므로, 공간이 협소하거나 사용자가 벽 가까이 활동하는 상황에서는 의도와 관계없이 반복적으로 개입하여 몰입을 저하시킬 수 있다.

또한 사전 설정된 정적 경계만으로는 활동 영역에 새로 진입한 사람이나 반려동물처럼 움직이는 객체에 충분히 대응하기 어렵다. 같은 거리에 있는 객체라도 사용자에게 접근하는 경우, 옆으로 통과하는 경우, 멀어지는 경우의 위험은 서로 다르다. 따라서 객체의 존재 여부 뿐 아니라 시간에 따른 위치 변화, 접근 추세, 충돌까지의 예상 여유 시간 및 사용자의 시야 내 방향을 함께 고려할 필요가 있다.

최근 Quest 계열 기기에서 Passthrough Camera를 컴퓨터 비전 입력으로 활용할 수 있는 환경이 마련되면서, HMD 자체에서 동적 객체를 인식하고 위험도를 계산하는 구조를 실험할 수 있게 되었다. 본 프로젝트는 정적 경계, 사용자 상태 및 동적 객체 위험을 통합하고, 위험 상황에 필요한 현실 정보만 선택적으로 제공하여 안전성과 몰입감 사이의 균형을 찾는 것을 목표로 한다.

1.3 프로젝트 목표

전체 과제의 목표는 다음과 같다.

- HMD 센서, 경계 정보 및 Passthrough Camera 영상을 이용한 상황 데이터 수집
- 정적 경계 위험, 사용자 움직임 상태 및 동적 객체 위험의 실시간 산출
- 위험 수준과 상황 맥락에 따른 선택적 Passthrough 시각화
- 사용자 반응 및 작동 이력의 수집과 개인화 제어 가능성 검토
- Guardian, Full Passthrough 및 제안 방식 사이의 안전성·몰입감 비교 평가

중간보고 시점의 세부 목표는 동적 객체 탐지 파이프라인의 기준선을 확보하는 것이다. 구체적으로 카메라 입력, 사람 검출, 객체 추적, 상대 위치 및 접근 상태 추정, 규칙 기반 위험도 계산과 외부 출력까지의 End-to-End 흐름을 구현하고 향후 Quest·Unity 및 상황별 시각화 실험으로 확장 가능한 구조를 마련한다.

제2장 전체 시스템 개요

2.1 시스템 처리 흐름 및 위험도

전체 시스템은 그림 2.1과 같이 실시간 위험 판단 루프와 세션 로그 기반 개인화 루프로 구성된다. 실시간 루프에서는 HMD 및 Controller 센서, Guardian 또는 공간 경계 정보, Passthrough Camera 영상을 수집하여 정적 경계 위험, 사용자 움직임 상태 및 동적 객체 위험을 계산한다. 계산된 위험 요소를 통합하여 최종 위험도를 산출하고, 이에 따라 Directional Passthrough, Augmented Virtuality, Volumetric Cue 또는 Full Passthrough 중 실험 조건에 해당하는 시각화 방식을 적용한다. 또한 세션 로그 기반 개인화 루프에서는 장기적으로 축적된 사용자 데이터를 활용하여 위험도 계산에 사용되는 가중치 (w_x)와 시각화 전환 임계값을 사용자에게 최적화하여 조정한다.

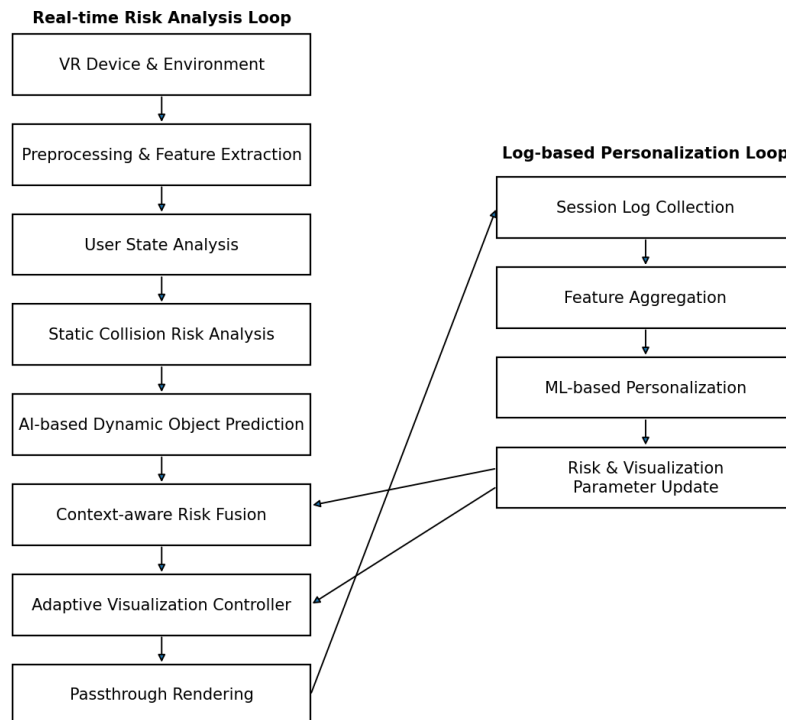


그림 2.1: 전체 시스템 처리 흐름

최종 위험도는 정적 경계 위험, 사용자 움직임 상태, 동적 객체 위험 및 상호작용 의도를 종합하여 계산하는 것을 목표로 했다. 위험도는 다음과 같이 정의한다.

$$R_{\text{total}} = w_{\text{static}}R_{\text{static}} + w_{\text{state}}R_{\text{state}} + w_{\text{dynamic}}R_{\text{dynamic}} + w_{\text{intent}}R_{\text{intent}}, \quad \sum_j w_j = 1. \quad (2.1)$$

여기서 R_{static} 은 정적 경계와의 관계를 기반으로 계산한 위험도, R_{state} 는 사용자의 속도, 가속도 등 움직임 상태를 기반으로 계산한 위험도, R_{dynamic} 은 동적 객체와의 충돌 위험도, R_{intent} 은 사람의 접근 및 상호작용 맥락을 반영한 위험도를 의미하며, w_j 는 각 위험 요소의 중요도를 반영하는 가중치이다.

2.2 시스템 레이어 정의

전체 시스템은 기능에 따라 VR Client, AI/ML, Backend, Data, Dashboard의 다섯 개 레이어로 구성된다. 각 레이어의 역할은 표 2.1과 같다.

표 2.1: 전체 시스템 레이어

레이어	주요 기술	역할
VR Client	Unity, C#, Meta XR SDK	센서 및 카메라 입력, 위험도 반영, Passthrough 제어
AI/ML	YOLOv8n, ONNX, 추적 및 시계열 모델	동적 객체 탐지·추적, 위험 예측 및 개인화 모델 수행
Backend	FastAPI	세션 관리, 로그 저장, 설정 및 결과 API 제공
Data	PostgreSQL	사용자 및 세션 로그, 실험 파라미터 저장
Dashboard	React	위험도 타임라인, 세션 정보 및 실험 결과 시각화

제3장 요구조건 및 제약 사항 분석에 대한 수정사항

3.1 정적 경계 및 사용자 상태 설계에 대한 수정사항

3.1.1 정적 경계 데이터

초기에는 Guardian Boundary 정보를 이용하여 사용자와 경계 사이의 거리를 계산하는 방식을 고려했다. 그러나 실제 Quest 개발 환경에서 Guardian Boundary API는 최신 SDK 및 OpenXR 환경에서 일부 기능이 제한적이며, 벽의 위치와 방향 정보를 직접적으로 얻기 어렵다는 문제가 있었다. 또한 단순 경계선 정보만으로는 사용자가 실제 벽면을 향해 접근하는지, 또는 벽에서 멀어지는지를 판단하기 어려웠다.

이에 따라 정적 경계 데이터 수집 방식을 Guardian Boundary 중심에서 Scene API 기반 Room Scene 정보 활용 방식으로 수정했다. 현재 구현에서는 Quest의 Space Setup으로 생성된 Room Scene 정보를 불러오고, WallFace, InvisibleWallFace 앵커만 필터링하여 실제 벽 정보를 사용한다. 각 벽은 위치와 법선 벡터를 갖는 평면으로 처리하며, HMD 위치와 가장 가까운 벽 사이의 거리 $d_{\min}$ 을 계산하도록 변경했다.

3.1.2 위험도 명칭 변경

착안 보고서에서는 벽이나 경계와의 충돌 위험도를 $R_{\text{collision}}$ 으로 표기했다.

$$R_{\text{total}} = w_c R_{\text{collision}} + w_s R_{\text{static}} + w_d R_{\text{dynamic}} + w_i R_{\text{intent}}, \quad \sum_j w_j = 1. \quad (3.1)$$

그러나 해당 명칭은 향후 동적 객체 충돌 위험도까지 포함하는 개념으로 오해될 수 있었다. 또한 사용자 움직임 상태 기반 위험도와 정적 경계 위험도가 명확히 분리되지 않는 문제가 있었다.

그러므로 위 문제를 해결하기 위해 기존의 $R_{\text{collision}}$ 은 정적 환경으로부터 발생하는 위험도를 의미하도록 R_{static} 으로 변경했으며, 기존의 R_{static} 은 사용자의 속도, 가속도 등 움직임 상태를 기반으로 계산되는 위험도를 의미하도록 R_{state} 로 변경했다.

명칭 변경 후의 계산식은 다음과 같다.

$$R_{\text{total}} = w_{\text{static}}R_{\text{static}} + w_{\text{state}}R_{\text{state}} + w_{\text{dynamic}}R_{\text{dynamic}} + w_{\text{intent}}R_{\text{intent}}, \quad \sum_j w_j = 1. \quad (3.2)$$

3.2 동적 객체 탐지 및 위험도 설계에 대한 수정사항

3.2.1 구현 범위와 출력의 재정의

착수보고서에서는 Passthrough Camera에서 사람·반려동물·이동 물체를 탐지하고 SORT/ByteTrack과 TCN-GRU로 R_{dynamic} 및 R_{intent} 를 예측하도록 계획했다. 중간 구현은 범위를 줄여 USB/노트북 카메라의 사람 전신 검출, 중심점 추적, 접근 상태·TTC 근사, 규칙 기반 위험도 및 JSON 출력까지를 MVP로 정의했다. 현재 점수는 전체 위험도 R_{total} 이 아니라 초기 기준선인 $R_{\text{dynamic_rule}}$ 이며, pose와 의도 label이 없어 R_{intent} 는 산출하지 않는다.

중간 구현의 입력은 연속 RGB 프레임이며 탐지 대상은 COCO class 0인 person이다. 얼굴이 아니라 사람 전체 bounding box를 사용한다. 전신 box가 화면 내 상대 위치, 크기 변화 및 접근 추세를 계산하는 데 더 적합하기 때문이다. 객체별 출력은 trackId, bbox, 화면 구역, 전방 상대 방향, 거리 구간, 접근 상태, TTC 근사값, 충돌 경로 및 위험 점수로 구성한다.

3.2.2 데이터 정의와 측정 제약

표 3.1: 동적 객체 입력 데이터의 정의와 한계

데이터	정의 및 산출 방법	해석 한계
검출·추적	YOLOv8n person 전신 box와 중심점 기반 trackId	교차·장시간 가림 시 ID 변경 가능
위치·거리	화면 좌/중/우, 전방 방향, box 면적 기반 far(< 0.03)/mid(< 0.06)/near	실제 공간 좌표나 미터 단위가 아님
접근 상태	최근 1.5초 $\sqrt{wh}$ 변화율; 0.04 이상 approaching, -0.04 이하 receding	최소 4회·250 ms 관측 필요, 자세·카메라 이동의 영향
TTC·경로	approaching일 때 $TTC_{\text{approx}} = 1/\text{scaleRate}$, 화면 중앙성으로 충돌 경로 추정	실제 물리 TTC와 사용자 이동 벡터가 아님
위험도	거리·접근·TTC·중앙 경로·객체 종류를 결합한 0-1 점수	실험 보정 전 휴리스틱 값

화면 가로 구역은 $c_x < 0.4$ 이면 left, $0.4 \leq c_x \leq 0.6$ 이면 center, $c_x > 0.6$ 이면 right로 분류한다. 단일 전방 카메라의 제약에 따라 사용자 기준 방향은 front-left, front-center, front-right로 표현한다. 대략적인 bearing은 $(c_x - 0.5) \times 150$ 으로 계산하며 음수는 왼쪽, 양수는

오른쪽이다.

접근 판정은 최소 4개 관측과 250 ms 이상의 이력이 쌓인 뒤 시작한다. 이력이 부족하면 unknown, 변화율이 두 임계값 사이이면 steady로 분류한다. TTC는 approaching일 때만 계산한다. 거리와 TTC는 단안 영상 기반 proxy이므로 실제 미터 거리나 물리적 충돌 시간으로 표현하지 않는다.

3.2.3 Feature 선정 근거

Feature는 “가까운가, 가까워지는가, 충돌 경로인가, 대응 시간이 얼마나 남았는가”를 설명하도록 거리 proxy, box 팽창률, TTC proxy, 화면 중앙성을 중심으로 선정했다. 단안 영상의 시간 변화로 TTC를 예측할 수 있다는 연구를 근거로 사용하되 실제 물리량으로 과장하지 않는다[1].

Feature는 단순히 구현 가능한 값이 아니라 동적 위험을 설명할 수 있는 값으로 제한한다. 거리와 접근 변화율은 현재 위험 수준과 증가 추세를 구분하고, TTC는 회피에 남은 시간, 화면 중앙성은 충돌 경로 가능성을 설명한다. Detector confidence는 최종 점수를 낮추는 보정값으로 사용한다. 객체 종류는 현재 person 단일 class이므로 다중 객체 확장 전까지 핵심 설명변수로 해석하지 않는다.

3.3 로그 기반 개인화 시스템 설계 수정사항

3.3.1 저장 세션 로그 항목 수정 및 Feature 재정의

착수보고서 단계에서는 세션 누적 시간(T_{session})이 길어질수록 사용자의 피로 및 둔감화로 인해 Passthrough 지속 시간이 짧아질 것이라는 가정을 바탕으로 $T_{\text{session_norm}}$ 을 주요 개인화 Feature로 포함했다. 그러나 이는 실제 사용자 행동을 뒷받침할 객체적 근거가 부족하며, 피로도가 개인화 판단에 미치는 영향의 인과관계가 불분명하다는 피드백을 받았다.

이에 본 프로젝트에서는 개인화 Feature의 선정 기준을 단순히 “기기에서 획득 및 구현 가능한 값”에서 “사용자의 실제 위험 판단 행동 및 물리적 위험을 설명할 수 있는 학술적 근거를 갖춘 값”으로 재정의하고 요구조건을 수정했다. 각 Feature는 HCI, 추천 시스템, 그리고 VR 근접학(Proxemics) 분야의 이론적 배경에 따라 다음과 같이 행동적 피드백, 물리적 요인, 실험적 요인의 세 가지 범주로 재분류했다.

- 행동적 피드백 변수 (Implicit/Explicit Feedback)

- r_{cancel} (수동 해제 비율): 사용자가 자동 활성화된 Passthrough를 수동으로 즉시 끈 행동의 비율이다. 이는 정보 검색 및 추천 시스템 분야에서 정의하는 가장 명시적

인 부정적 피드백(Negative Feedback) 신호이며[5], 시스템의 개입이 불필요했음을 나타내는 가장 신뢰도 높은 행동 지표이다.

- t_{pt_bar} (평균 지속 시간): 사용자가 별도의 조작 없이 Passthrough 시각화를 유지한 시간이다. 사용자 행동 분석 연구에서의 체류 시간(Dwell Time) 개념과 유사하게, 이 값이 길어질수록 사용자가 현재 제공된 현실 환경 정보를 유용하게 인지하고 수용(Positive Feedback)했음을 암묵적으로 의미한다[6].
- f_{pt} (활성화 빈도): 특정 세션 내에서 Passthrough가 활성화된 빈도이다. 이는 사용자의 주관적 선호보다 해당 공간 환경의 장애물 밀도와 정적 위험 노출 빈도를 대변하는 환경 Baseline 성격의 지표로 활용된다.

● 물리적 위험 변수 (Physical Risk Factors)

- v_{h_bar}, v_{h_max} (평균/최대 head speed): 가상 환경 내 사용자의 물리적 활동성을 나타낸다. 사용자의 회전 및 이동 속도가 빠를수록 잠재적 충돌 발생 시 제동 거리와 회피 반응 시간이 더 많이 요구되므로, 물리적 위험도와 양의 상관관계를 갖는 물리적 행동 신호이다.
- A_{space_norm} (공간 크기): 가상 경계가 설정된 실제 물리 공간의 면적이다. VR 공간 인지 및 안전 경계 연구(Confined Spaces VR Boundary)[7]에 따르면, 공간이 협소할수록 물리적 장애물과의 조우 확률이 기하급수적으로 증가하며, 이는 가상 환경 내에서의 심리적 위축과 수동 해제 패턴 변화로 이어진다.

● 실험적 변수 (Experimental Factors)

- $T_{session_norm}$ (누적 세션 시간): 기존의 근거 없는 “피로도” 가정은 완전히 제거한다. 다만, 장시간 세션이 지속됨에 따라 사용자가 가상 환경에 완벽히 적응하여 경계심이 완화되거나, 반복적인 시각화 개입에 무더지는 알람 피로(Alarm Fatigue) 및 둔감화(Habituation) 현상[8]과 연관될 가능성이 존재한다. 따라서 이를 “근거 미확정 변수”로 분류하고, 실제 사용자 대상의 실험 데이터를 확보한 뒤 Random Forest 모델의 변수 중요도(Feature Importance) 분석을 통해 최종 유지 또는 영구 탈락 여부를 결정하기로 설계했다.

결과적으로, 직관에 의존하던 기존의 설계를 전면 수정하여 개인화 Feature들을 사용자의 실제 인지 반응을 대변하는 **행동적 피드백 측면**(r_{cancel}, t_{pt_bar})과 안전 마진 확보를 위한 **물리적 운동 데이터 측면**($v_{h_bar}, A_{space_norm}$)으로 이원화했다. 또한 타당성이 검증되지 않은 변수는 실험적 검증 파이프라인을 거치도록 설계하여 개인화 모델의 신뢰성과 안전성을 동시에 확보했다.

제4장 설계 상세화 및 변경 내역

4.1 정적 경계 및 사용자 상태 설계 상세화

4.1.1 정적 경계 위험도 설계

정적 경계 위험도 R_{static} 은 사용자가 벽이나 고정 경계에 가까워질 때 발생할 수 있는 충돌 위험을 수치화한 값이다. 기존에는 벽과의 거리만을 중심으로 판단할 수 있었으나, 거리만 사용하는 경우 벽 근처에 정지해 있는 상황과 벽을 향해 빠르게 이동하는 상황을 구분하기 어렵다. 따라서 거리, TTC, 접근 가속도, 사각지대 위험도를 함께 고려하도록 설계를 상세화했다.

$$R_{static} = \frac{w_d R_d + w_{TTC} R_{TTC} + w_a R_a + w_{blind} R_{blind}}{w_d + w_{TTC} + w_a + w_{blind}} \quad (4.1)$$

각 Feature의 의미와 설계 근거는 다음과 같다.

표 4.1: 정적 경계 위험도 Feature와 설계 근거

Feature	의미	설계 근거
R_d	거리 기반 위험도	벽과 가까울수록 충돌 가능성이 높아지므로 가장 기본적인 정적 경계 위험 요소로 사용했다.
R_{TTC}	충돌 예상 시간 위험도	같은 거리라도 빠르게 접근하는 경우 더 위험하므로, 현재 속도로 벽에 도달하기까지의 시간을 반영했다.
R_a	접근 가속도 위험도	벽 방향으로 급격히 가속하는 상황은 가까운 미래의 충돌 가능성을 높이므로 반영했다.
R_{blind}	사각지대 위험도	사용자가 벽을 정면으로 보고 있는지, 측면 또는 후방에 있어 인지하기 어려운지를 반영했다.

현재 기본 가중치는 거리 위험도 0.30, TTC 위험도 0.30, 접근 가속도 위험도 0.25, 사각지대 위험도 0.15로 설정했다. 거리와 TTC는 충돌 가능성과 직접적으로 연결되므로 상대적으로 높은 가중치를 부여했고, 접근 가속도는 갑작스러운 움직임을 보완적으로 반영하기 위해 사용했다. 사각지대 위험도는 사용자의 인지 가능성을 보조적으로 반영하는 요소이므로 상대적으로 낮은 가중치를 부여했다. 모든 가중치와 기준값은 Unity Inspector에서 수정

가능하도록 구성했다.

4.1.2 사용자 상태 위험도 설계

사용자 상태 위험도 R_{state} 는 HMD의 속도, 가속도, 각속도를 기준으로 사용자의 움직임 상태를 수치화한 값이다. 정적 경계 위험도가 벽과의 관계를 나타낸다면, 사용자 상태 위험도는 사용자가 현재 얼마나 안정적으로 움직이고 있는지를 나타낸다.

$$R_{state} = \begin{cases} 0.0, & \text{Static} \\ 0.5, & \text{Dynamic} \\ 1.0, & \text{Agitated} \end{cases} \quad (4.2)$$

현재 기준값은 정적 상태에 대해 $v_h < 0.05 m/s$, $a_h < 0.3 m/s^2$, $\omega_h < 0.3 rad/s$ 로 설정했다. 격동 상태는 $v_h > 1.0 m/s$, $a_h > 5.0 m/s^2$, $\omega_h > 2.5 rad/s$ 중 하나를 만족할 때로 설정했다. 이는 센서 노이즈 수준의 미세한 움직임과 실제 이동을 구분하고, 빠른 이동이나 급격한 회전을 우선적으로 위험 상태로 감지하기 위한 초기 기준이다.

4.2 동적 객체 탐지 및 위험도 설계 상세화

4.2.1 모듈 구조와 처리 흐름

현재 처리 흐름은 다음과 같다.

```
CameraSource → OnnxPersonDetector → SimpleTracker → Motion/LocationEstimator
→ WeightedRiskEstimator → JSON/NDJSON
```

각 단계는 인터페이스로 분리하여 향후 Quest CameraSource, ByteTrack, depth 기반 위치 추정기 및 Unity 전송기로 교체할 수 있다.

모듈별 역할은 다음과 같다.

- MockCameraSource는 정의된 Detection 프레임을 생성하고, OpenCVCameraSource는 USB 카메라 BGR 프레임을 수집한다.
- FakeDetector는 Mock Detection을 반환하며, OnnxPersonDetector는 YOLOv8n ONNX로 사람 전신을 검출한다.
- SimpleTracker는 같은 label의 중심점 거리로 ID를 유지한다.

- **HistoryMotionEstimator**는 최근 1.5초 이력의 변화율 중앙값과 평활화로 접근·정지·후퇴 및 TTC를 추정한다.
- **RelativeLocationEstimator**는 화면 구역, 전방 방향, 거리 구간과 bearing을 계산한다.
- **WeightedRiskEstimator**는 Feature를 결합하고 Publisher는 터미널과 최신·누적 JSON 파일에 결과를 기록한다.

검출 box는 원본 영상 좌표로 복원한 뒤 0-1 범위로 정규화한다. 출력은 마지막 결과의 터미널 JSON, `outputs/latest.json`, 프레임별 `outputs/risk.ndjson`으로 구분한다.

표 4.2: 착수 계획 대비 주요 변경

항목	착수 계획	중간 구현 및 변경 이유
입력·대상	Quest, 다중 객체	USB 카메라와 사람 전신으로 제한해 end-to-end 흐름 우선 검증
추적·거리	ByteTrack, 실제 거리	SimpleTracker와 box 면적 proxy로 인터페이스 기준선 확보
움직임 예측 모델	box 크기 변화 TCN-GRU+규칙	근접 시 높이 포화를 줄이도록 $\sqrt{wh}$ 의 최근 이력 사용 label 부재로 설명 가능한 규칙 기반 baseline 우선 구현
출력	Unity/Backend 연동	<code>latest.json</code> , <code>risk.ndjson</code> 으로 데이터 계약 우선 검증

SimpleTracker는 데이터 흐름을 검증하기 위한 기준선이다. 사람이 서로 교차하거나 가림이 길어지면 ID가 바뀔 수 있고, 속도 예측과 Appearance Feature가 없다. 따라서 교차·가림 상황의 ID switch를 측정된 뒤 SORT 또는 ByteTrack으로 교체한다. 실제 거리 역시 depth 또는 카메라 보정 정보를 확보하기 전까지 `distanceBand`로 제한한다.

4.2.2 규칙 기반 위험도 설계

현재 위험도는 다음과 같이 계산한다.

$$R_{\text{dynamic_rule}} = 0.30R_{\text{proximity}} + 0.30R_{\text{approach}} + 0.15R_{\text{TTC}} + 0.10R_{\text{path}} + 0.05R_{\text{type}} + 0.10R_{\text{proximity}}R_{\text{approach}}.$$

검출 confidence를 반영하고 receding이면 0.55를 곱한다. 단계는 safe(< 0.25), caution(< 0.50), warning(< 0.75), danger(≥ 0.75)이며, 가중치와 임계값은 향후 실험으로 보정해야 한다.

4.3 로그 기반 개인화 시스템 설계 상세화

본 프레임워크의 개인화 제어 루프는 일회성 예측에 그치지 않고, 사용자의 행동 피드백을 실시간으로 수집하여 시스템 파라미터에 지속적으로 반영하는 선순환 구조를 갖는다.

4.3.1 로그 수집 및 버퍼 적재 및 Feature 변환

사용자가 가상 환경에 진입하면 공간 크기($A_{\text{space_norm}}$)와 누적 세션 시간($T_{\text{session_norm}}$)이 세션 단위 로그로 적재된다. 이후 Passthrough 개입 이벤트가 트리거될 때마다 수동 해제 여부($cancel_{\text{manual}}$), 지속 시간(t_{duration}), 헤드 트래킹 기반 물리 속도가 이벤트 버퍼에 순차 적재된다.

이 버퍼는 크기 W , 이동 간격 S 의 슬라이딩 윈도우로 슬라이싱되며, 윈도우가 이동할 때마다 원시 로그는 평균값($t_{\text{pt_bar}}, v_{\text{h_bar}}$)과 최댓값($v_{\text{h_max}}$) 등으로 정제되어 7차원 Feature 벡터 $\mathbf{x}$ 가 된다. W, S 의 구체적 값은 실 데이터로 검증되지 않은 잠정치이며, 추후 사용자 테스트를 통해 재조정할 예정이다.

4.3.2 실버 라벨링 및 온디바이스 모델 학습

사용자의 명시적 조작이 없는 가상 환경의 특성을 고려하여, 다음과 같은 암묵적 행동 기반의 실버 라벨링 규칙을 정의하여 지도 학습을 수행한다.

$$\text{Label}(e) = \begin{cases} 0 & \text{(Negative), if } cancel_{\text{manual}} = 1 \text{ and } t_{\text{dur}} < 2.0 \text{ s} \\ 1 & \text{(Positive), if } cancel_{\text{manual}} = 0 \text{ and } t_{\text{dur}} \geq 3.0 \text{ s} \\ \text{Neutral,} & \text{otherwise (경계 구간 완충 및 제외)} \end{cases} \quad (4.3)$$

라벨링의 근거인 Claypool et al.[6]은 웹페이지 체류 시간 (Dwell Time)이 명시적 관심도와 강한 상관관계를 가짐을 실증하였고, 이는 추천 시스템 분야에서 체류 시간을 암묵적 피드백으로 채택하는 근거가 되었다. 본 프레임워크는 이를 Passthrough 상호작용에 적용한다: 별도 해제 없이 일정 시간 이상 유지되면 암묵적 수용, 즉각적인 수동 해제는 명시적 거부로 해석한다.

다만 기준값 2.0초/3.0초 자체는 선행 연구에서 도출된 수치가 아니라, mock 데이터 실험과 구현상의 편의를 고려한 경험적 설계값(heuristic)임을 명시한다.

모델은 분류 성능과 해석 가능성이 높은 Random Forest를 채택하였으며, Unity Sentis 온디바이스 추론을 위해 ONNX로 변환하였다. Sentis 런타임에서 ONNX 모델은 사용자의 최신 상태 벡터 $\mathbf{x}$ 를 입력받아 실시간으로 부정적 피드백 확률 P_{neg} 를 추론한다.

4.3.3 연속적 파라미터 매핑 및 실시간 피드백

Cold-Start 기준을 충족한 사용자에게 대해, P_{neg} 의 최근 윈도우 평균값 $\bar{P}_{\text{neg}}$ 을 스케일링 팩터로 삼아 통합 위험 가중치 w_c 와 시각화 반응 임계값 τ_{viz} 를 이산적 전환이 아닌 연속 함수로 갱신한다.

$$w_c = w_{c,\text{base}} \cdot (1.0 - \alpha \cdot \bar{P}_{\text{neg}}) \quad (4.4)$$

$$\tau_{\text{viz}} = \tau_{\text{base}} \cdot (1.0 + \beta \cdot \bar{P}_{\text{neg}}) \quad (4.5)$$

계단식 제어는 확률값이 임계점을 넘는 순간 급격히 전환되어 유사한 상황에서도 결과가 불연속적으로 달라지는 문제가 있어, $\bar{P}_{\text{neg}}$ 변화에 비례한 연속적 조정을 채택하였다. 조정 강도 α, β 는 현재 임의로 설정한 잠정치이며 추후 사용자 테스트로 튜닝이 필요하다.

$\bar{P}_{\text{neg}}$ 가 상승하면 w_c 는 낮추고 τ_{viz} 은 높여 불필요한 노출을 줄인다. 갱신된 값은 Unity 클라이언트의 Context-aware Risk Fusion 및 Adaptive Visualization Controller로 전달되고, 그 결과 발생하는 새로운 사용 이력은 다시 3.4.1절의 로그로 수집되어 개인화 루프의 폐쇄 루프(Closed Loop) 구조를 완성한다.

제5장 프로젝트 진척도 및 수행 내용

5.1 구성원별 진척도

각 구성원은 착수보고서에서 정의한 담당 영역을 중심으로 프로토타입 단위 구현을 진행하였으며 세부 진척 내역은 아래 표 5.1과 같다.

구성원	진척도 요약
최아영	Quest 개발 환경 구축 및 공간 데이터 연동, HMD 및 Controller 기반 센서 데이터 수집, Feature 알고리즘 및 위험도 계산 로직 구현
이승주	인터페이스 기반 MVP, Mock/USB 입력, 사람 검출·추적·움직임·위험도·JSON 출력 구현 (16개의 자동 테스트를 통과).
따다소	Mock 세션/이벤트 로그 생성 및 Feature 추출 파이프라인 구현, Random Forest 모델 학습 및 ONNX 변환(확률값 일치 검증 완료), Cold-start 판단 로직 및 개인화 가중치 계산 로직 구현

5.1.1 최아영

표 5.1: 정적 경계 및 사용자 상태 설계 파트 진척도

항목	진행 상태
Quest 개발 환경 구축	Unity, C#, Meta XR SDK 기반 Quest 실행 환경 구성을 완료했다.
Scene API 기반 공간 정보 수집	Room Scene 데이터를 로드하고 WallFace 및 InvisibleWallFace 앵커를 필터링하는 기능을 구현했다.
HMD 및 컨트롤러 Feature 계산	HMD의 속도, 가속도, 각속도와 좌·우 컨트롤러 속도, Hand/-Head Ratio 계산을 구현했다.
벽 접근 Feature 계산	최근접 벽 거리, 벽 방향 접근 속도, 접근 가속도, TTC(Time-To-Collision), 시선-벽 방향 각도 계산을 구현했다.
정적 경계 위험도 계산	R_d , R_{TTC} , R_a , R_{blind} 을 이용한 R_{static} 계산을 구현했다.
사용자 상태 위험도 계산	Static, Dynamic, Agitated 상태 분류 및 R_{state} 계산을 구현했다.
최종 위험도 및 판단 결과 출력	R_{total} , 위험 단계 및 Passthrough ON/OFF 판단 결과를 UI에 출력하도록 구현했다.

5.1.2 이승주

표 5.2: 동적 객체 인식 및 위험도 판단 파트 진척도

항목	진행 상태
Python 실행 환경 구성	ml-dynamic-object Python 패키지 구조를 구성하고 Mock-/USB 입력 경로를 포함한 실행 환경을 구축했다.
사람 객체 검출	YOLOv8n ONNX 모델 기반 사람 전신 검출 기능을 구현하고, 검출 결과를 기반으로 객체 정보를 추출하도록 구성했다.
객체 ID 추적	검출된 사람의 중심점 좌표를 활용하여 프레임 간 객체 ID를 유지하는 추적 기능을 구현했다.
움직임 상태 판단	최근 1.5초 동안의 객체 위치 이력을 기반으로 approaching, steady, receding 상태를 분류하도록 구현했다.
접근 위험 Feature 계산	객체의 상대 위치와 이동 정보를 기반으로 TTC(Time-To-Collision) 및 접근 방향 관련 Feature 계산 기능을 구현했다.
검출 노이즈 보정	느린 접근 상황 및 순간적인 Bounding Box 변화에 대응하기 위해 $\sqrt{wh}$ 변화율, 중앙값 기반 필터링 및 평활화 기법을 적용했다.
위험도 판단 로직	규칙 기반 위험 단계 분류 로직을 구현하여 객체 접근 상태에 따른 위험도를 판단하도록 구성했다.
결과 출력 및 데이터 저장	터미널 JSON 출력, latest.json, risk.ndjson 기반의 위험도 결과 저장 기능을 구현했다.

5.1.3 파다소

표 5.3: 로그 기반 개인화 시스템 설계 및 모델링 파트 진척도

항목	진행 상태
Python 실행 환경 구축	ml-personalization 패키지 설계 및 피드백 검증용 Mock 로그 생성기 구현 완료
개인화 Feature 추출	HCI 체류 시간 이론[6] 기반 수동 해제율, 활동성(Head speed), 공간 크기 등 7차원 입력 벡터 변환 알고리즘 구현
실버 라벨링 규칙 설계	사용자 거부 및 수용 행동 기반의 암묵적 의사 라벨링 규칙(Silver Labeling) 수립 및 데이터 정제 필터 적용
경량 머신러닝 모델링	연산 효율성과 해석 가능성을 확보한 Random Forest 기반 개인화 모델 설계 및 학습 파이프라인 완성
ONNX 변환 및 검증	모델의 ONNX 변환 및 Unity Sentis 이식 완료
연속적 파라미터 매핑	시각적 이질감 방지를 위해 Negative 확률(P_{neg})을 매핑 인자로 삼는 연속형 위험 가중치(w_c) 및 임계값(τ_{viz}) 제어식 구현
Cold-Start 대응 로직	초기 세션 수($N_{min_sessions}$) 기준 미달 시 시스템 안전성 확보를 위한 Baseline 가중치 강제 적용 로직 구현

5.2 보고 시점까지의 과제 수행 내용 및 중간 결과

Quest 환경에서 정적 경계 및 사용자 상태 위험도 계산이 실제로 동작하는지 확인했다. 현재 구현 화면에서는 좌측 UI에 HMD 위치, 최근접 벽 번호, 벽까지의 거리, HMD 속도, 가속도, 각속도, 좌우 손 속도, 손 평균 속도, Hand/Head Ratio, 벽 방향 접근 속도, 접근 가속도, TTC, 벽 접근 여부가 표시된다. 우측 UI에는 사용자 상태, R_{state} , R_d , R_{TTC} , R_a , R_{blind} , R_{static} , R_{total} , 위험 단계, Passthrough threshold, Passthrough 판단 결과가 표시된다.

중간 실행 결과의 한 예로, 사용자가 가장 가까운 벽에서 약 $0.809m$ 떨어진 상태에서 사용자 상태는 Dynamic으로 분류되었고 $R_{state} = 0.50$ 으로 계산되었다. 해당 시점의 R_{static} 은 약 0.10, R_{total} 은 약 0.26으로 나타났다. 이는 Passthrough threshold 0.60보다 낮기 때문에 Passthrough Decision: OFF로 표시되었다.

그림 5.2를 통해 현재 프로토타입이 Quest 환경에서 실시간으로 Feature를 계산하고, 위험도 및 Passthrough 판단 결과를 시각적으로 출력할 수 있음을 확인했다.

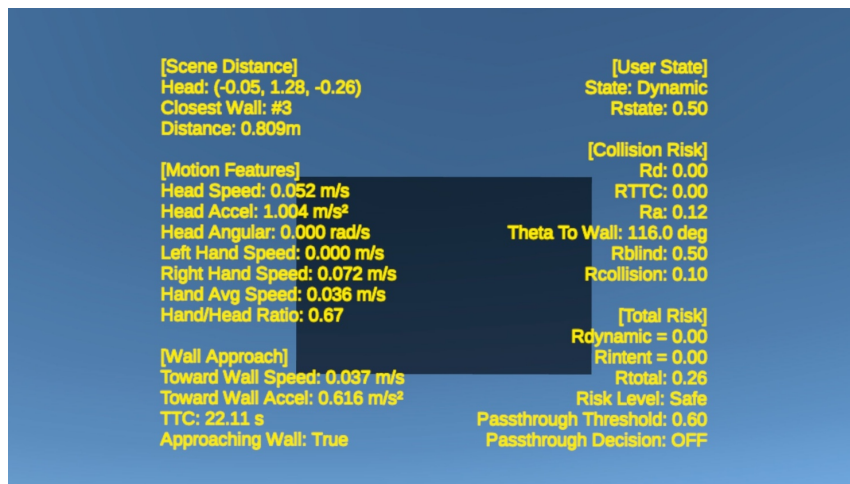


그림 5.1: Quest 환경에서의 실행 결과 화면

동적 객체 탐지 및 위험도 설계 부분 같은 경우 pytest 결과 16개 테스트가 모두 통과했다. Mock 접근 시나리오에서 approaching과 위험도 증가를 확인했으며, USB 카메라에서는 사람 전신과 person / steady / caution / 0.44를 실시간 표시했다. 중간 결과의 의미는 영상 입력부터 검출, 추적, Feature, 위험도, 외부 출력까지 이어지는 비교 기준선을 확보했다는 것이다. 현재 한계는 Quest · depth · HMD 이동 보정 부재, 사람만 지원, 교차 · 가림에 취약한 추적, 물리 단위가 아닌 거리/TTC, 미구현 intent · TCN-GRU, 미검증 가중치, Unity 시각화 미연동이다.

마지막으로 세션 로그 기반 개인화 시스템에 관하여 Mock 세션/이벤트 로그 생성부터 Feature 추출, Random Forest 학습, ONNX 변환, Cold-start 판단, 개인화 가중치 계산까지



그림 5.2: USB 카메라 사람 검출 및 위험도 표시

전체 파이프라인을 end-to-end로 구현하여 정상 동작을 확인했다. ONNX 변환 모델은 클래스 예측과 확률값이 원본 sklearn 모델과 일치함을 검증했으며, 모델 크기는 0.09MB로 경량화 목표(50MB) 대비 충분한 여유를 확보했다.

다만 현재까지의 결과는 전부 Mock 데이터를 기반으로 하며, 라벨이 Feature로부터 규칙적으로 파생된 구조이기 때문에 나온 RF 모델 정확도(97%)는 파이프라인 정상 동작 확인 이상의 의미를 갖지 않는다. 또한 교수 피드백을 계기로 $T_{session_norm}$ 의 근거(피로도 가정)를 재검토했으며, 이는 실 데이터 확보 후 Feature Importance 분석을 통해 유지 여부를 다시 결정할 예정이다.

제6장 갱신된 과제 추진 계획

향후 개발은 시스템 통합, 성능 개선 및 사용자 실험을 중심으로 단계적으로 진행할 예정이다. 본 프로젝트의 전체 일정 및 개발 흐름은 아래의 간트 차트(그림 6.1)와 같다.

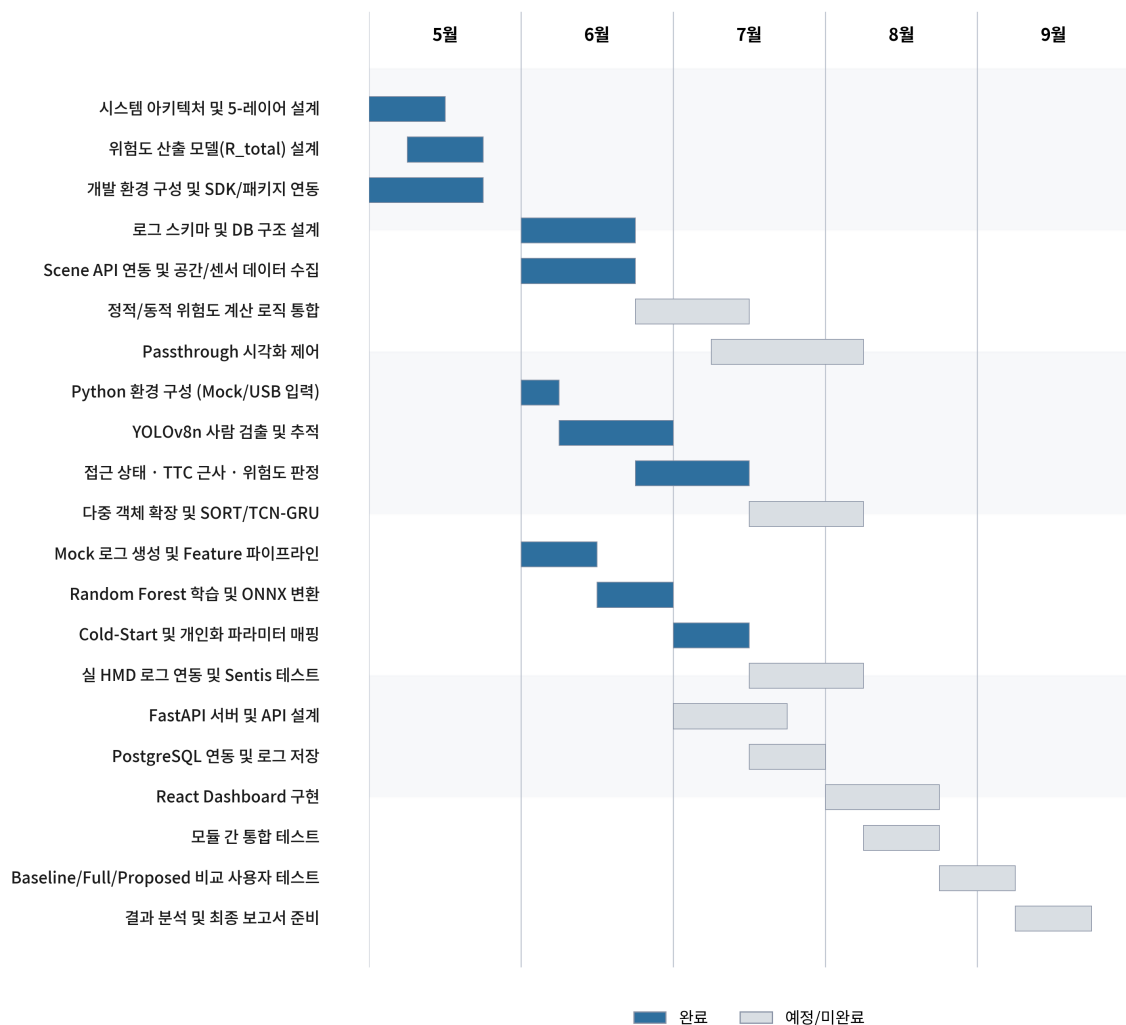


그림 6.1: 프로젝트 개발 일정 (중간보고서 기준 진척 반영)

6.1 시스템 통합 및 성능 개선

현재까지는 Unity, AI/ML, Backend 및 Database를 구성원별로 독립적으로 개발했다. 향후에는 각 모듈을 하나의 시스템으로 통합하여 실제 XR 환경에서 정상적으로 동작하는지 검증할 예정이다.

표 6.1: 향후 통합 및 개발 계획

개발 항목	향후 수행 내용
세션 로그 저장 구조 추가	Mock 데이터가 아닌 실제 HMD 및 Controller 데이터를 세션 단위 로그 형식으로 정리한다.
Unity-AI 연동	AI 모듈의 객체 탐지 및 위험도 계산 결과를 Unity로 전달하여 실시간 Passthrough 시각화와 연동한다.
Backend 통합	Backend와 Unity를 연동하여 사용자 정보, 세션 데이터 및 실험 결과를 저장하고 관리한다.
Meta Quest HMD 연동	Quest HMD의 카메라 및 센서 데이터를 실제 입력으로 사용하여 전체 시스템이 정상적으로 동작하는지 검증한다.
실시간 데이터 검증	실제 환경에서 객체 탐지, 위험도 계산 및 시각화 결과가 실시간으로 정상 동작하는지 확인한다.
위험도 계산 튜닝	위험도 계산에 사용되는 가중치와 임계값을 다양한 시나리오에서 반복적으로 조정하여 정확도를 개선한다.
시각화 전환 튜닝	위험 수준에 따른 Passthrough 전환 조건과 시각화 방식을 사용자 경험을 고려하여 최적화한다.
성능 최적화	모듈 간 통신 지연, 추론 속도 및 프레임 저하를 최소화하여 실시간성을 확보한다.
사용자 실험 환경 구축	비교 평가를 위한 실험 시나리오와 환경을 구성한다.
비교 평가 수행	Guardian, Full Passthrough 및 Adaptive Passthrough Framework를 비교하여 안전성과 사용자 경험을 평가한다.

우선 Meta Quest HMD와 연동하여 센서 정보와 Passthrough 영상이 Unity로 정상적으로 입력되는지 확인하고, AI 모듈에서 계산한 위험도와 객체 정보를 Backend를 통해 Unity와 연동하여 실시간으로 시각화가 이루어지는지 검증할 계획이다. 또한 통합 과정에서 각 모듈 간 데이터 송수신 지연, 시스템 안정성 및 실시간 처리 성능을 점검하고, 위험도 계산에 사용되는 임계값과 가중치, 시각화 전환 조건 등을 실제 환경에 맞게 조정할 예정이다. 이후 통합 시스템을 기반으로 다양한 시나리오에서 반복적인 테스트를 수행하여 성능을 개선하고, 사용자 실험을 위한 최종 시스템을 완성할 계획이다.

6.2 비교 평가 계획

본 프로젝트는 Adaptive Passthrough Framework의 효과를 검증하기 위해 기존 Guardian 및 Full Passthrough와 비교 평가를 수행할 예정이다. 또한 위험 상황에 따라 적용되는 Passthrough 시각화 방식이 안전성과 사용자 경험에 미치는 영향을 함께 분석하고자 한다.

평가는 동일한 XR 환경에서 다양한 위험 상황을 구성하여 진행할 예정이며, 안전성과 사용자 경험을 중심으로 시스템의 효과를 분석한다. 세부 실험 시나리오와 비교 조건은 구현 완료 후 관련 선행연구를 참고하여 최종 확정할 예정이다. 구체적인 평가 방법과 기준은 선행연구를 조사하여 최종보고서에서 상세히 제시할 예정이다.

참고문헌

- [1] A. Manglik et al., “Forecasting Time-to-Collision from Monocular Video,” arXiv:1903.09102, 2019.
- [2] J. von Willich et al., “You Invaded My Tracking Space!,” DIS, 2019.
- [3] D. Medeiros et al., “Promoting Reality Awareness in VR through Proxemics,” IEEE VR, 2021.
- [4] Y. Kudo et al., “Towards Balancing VR Immersion and Bystander Awareness,” PACM HCI, 2021.
- [5] T. Joachims et al., “Accurately interpreting clickthrough data as implicit feedback,” SIGIR, 2005.
- [6] M. Claypool et al., “Implicit interest indicators,” ACM IUI, 2001.
- [7] C. Fink et al., “CoVR: Dynamic virtual boundaries for safer VR experiences in confined spaces,” CHI, 2021.
- [8] J. P. Bliss, “Alarm mistrust in assistive technology,” Applied Ergonomics, 2003.
- [9] S. Mayer et al., “Anticipation Before Action: EEG-Based Implicit Intent Detection for Adaptive Gaze Interaction in Mixed Reality,” CHI, 2026.
- [10] H. Bharadhwaj et al., “Meta-Learning for User Cold-Start Recommendation,” IEEE Computer Society, 2024.